Building a POSIX Shell

Process Control, Pipes, and Signals in C++

A complete implementation study

of 1,841 lines of source

Advanced Operating Systems

Assignment 2 | Monsoon 2025

Md Taufique Hussain

About this book

Every behavioural claim in these pages was verified by running the shell, not inferred from reading the source. Build output, file permissions, history limits, and `ls` formatting were checked against the running program and against the system tools they imitate. Where the implementation falls short, the limitation is stated with the measurement that revealed it.

Typeset with XeTEX. Source: 17 `.cpp` files and one header, compiled with `-Wall -Wextra -std=c++11` and no warnings.

Contents

Preface **1**

1 The Shape of a Shell **3**
1.1 The core loop . . . 3
1.2 The one idea that explains the rest . . . 3
1.3 Project statistics . . . 4
1.4 File map . . . 4

2 Global State **7**
2.1 MyHomeDir is not $HOME . . . 7
2.2 PrevDir and the cd - target . . . 7
2.3 Why globals at all . . . 8

3 The Life of an Input Line **9**
3.1 Details that matter . . . 9

4 Parsing **11**
4.1 Stage one: splitting on semicolons . . . 11
4.2 Stage two: the character scanner . . . 12
4.3 Quote handling is deliberately shallow . . . 12

5 The Executor **15**
5.1 Recognising a builtin . . . 15
5.2 The router . . . 15
5.3 Redirection for builtins . . . 16

6 The Builtin Commands **17**
6.1 The prompt . . . 17
6.2 cd . . . 18
6.3 echo . . . 18
6.4 pwd . . . 18
6.5 ls . . . 18
6.6 pinfo . . . 19
6.7 search . . . 20

6.8 history . . . 21
6.9 jobs . . . 22

7 External Commands 23
7.1 Before the fork . . . 23
7.2 Inside the child . . . 23
7.3 Inside the parent . . . 24

8 Redirection 27
8.1 The open flags . . . 27
8.2 The dup2 pattern . . . 28

9 Pipelines 29
9.1 The six steps . . . 29

10 Signals and Job Control 31
10.1 Ctrl-C, or SIGINT . . . 31
10.2 Ctrl-Z, or SIGTSTP . . . 31
10.3 SIGCHLD, or reaping the background . . . 32
10.4 Ctrl-D . . . 32

11 Autocomplete 33
11.1 Deciding what to complete . . . 33
11.2 Where the candidates come from . . . 33
11.3 The generator protocol . . . 33

12 System Call Reference 35

13 Twelve Concurrency Traps 37

14 Test Evidence 39

15 Known Limitations 41

16 Rule Compliance 43

17 Viva Question Bank 45

Preface

This book documents a POSIX shell written in C++ for Advanced Operating Systems, Assignment 2, Monsoon 2025. It is not a summary. It walks through all 1,841 lines of the implementation — every module, every function, every system call, and every edge case that had to be reasoned about.

It was written to serve three purposes: as a record of what was built and why, as a revision text before a viva, and as a working explanation of how a Unix shell actually operates underneath the prompt.

Every claim about behaviour in these pages was verified by running the shell, not by reading the code and assuming. Where the implementation has a limitation, that limitation is stated plainly, with the measurement that revealed it and the clause of the assignment specification that permits it. Chapter 15 lists the evidence; Chapter 16 lists the limitations.

What was verified. A clean rebuild produces zero warnings under `-Wall -Wextra`. The `ls -l` output is byte-identical to the system `ls`, including the block total and column alignment. Redirected output files are created at mode 0644. History stores exactly 20 commands, displays 10, and survives across sessions. The forbidden functions `popen`, `pclose`, and `system` are absent from the source.

How to read it. Chapters 1 through 4 build the mental model: architecture, state, the lifecycle of one input line, and parsing. Chapters 5 through 12 are the implementation proper, one subsystem at a time. Chapters 13 and 14 are reference material — a system call index and the concurrency traps. Chapters 15 through 18 cover testing evidence, limitations, rule compliance, and a viva question bank.

If you are revising under time pressure, read Chapter 1, then Chapter 14, then Chapter 18.

1

The Shape of a Shell

1.1 The core loop

A shell is an infinite loop doing four things:

```
read a line
  -> split into commands
  -> decide builtin or external
  -> run it and wait (or not)
```

Everything else — pipes, redirection, signals, job control — hangs off step four. Strip those away and what remains is a program that reads text and starts processes.

1.2 The one idea that explains the rest

If you remember a single structural fact from this book, make it this one:

> **Builtin commands run inside the shell process. External commands run in a forked child.**

That distinction is not a style choice. It is forced by the semantics of `fork`, and it propagates into nearly every design decision in the codebase.

It explains why `cd` *must* be a builtin: `fork` gives a child a copy of the parent's state, including the working directory. A child calling `chdir` changes only its own copy, then exits. The shell — the process that actually needs to move — is untouched, and the user sees nothing happen.

It explains why builtins need a save-and-restore dance around their file descriptors instead of the simple `dup2` an external command uses. An external command's redirections are applied in a child that is about to be replaced by `execvp` and then discarded, so noth-

ing needs cleaning up. A builtin has no child to throw away; if it redirects its own stdout and does not put it back, the shell's prompt disappears into a file.

And it explains why a builtin inside a pipeline silently loses its side effects. Each pipeline stage is a forked child, so `cd sub | cat` changes a directory in a process that immediately exits. This is not a bug — bash does exactly the same — but it surprises people, and it follows directly from the same rule.

1.3 Project statistics

Property	Value
Language	C++ (C++11)
Source files	17 `.cpp` + 1 `.h`
Total lines	1,841
Compiler flags	`-Wall -Wextra -std=c++11 -g`
Warnings on clean build	**zero**
External dependency	GNU readline (`-lreadline`)
Binary size	~1.2 MB (with debug symbols)

1.4 File map

Every command lives in its own file. The assignment requires modular code, and one-command-per-file makes the mapping from specification requirement to source file immediate. It also keeps rebuilds small — the makefile declares a per-object rule, so editing `ls.cpp` recompiles only `ls.o`.

File	Lines	Responsibility
`functions.h`	79	Shared declarations, `Redirection` and `Job` structs, the two globals
`main.cpp`	81	REPL loop, readline call, `;` dispatch, `exit`, Ctrl-D
`parser.cpp`	103	Stage 1 split on `;`, Stage 2 tokenizer

File	Lines	Responsibility
`executor.cpp`	137	Router, builtin dispatch, builtin redirection wrapper
`display.cpp`	48	Builds the prompt
`cd.cpp`	61	`cd` with `.`, `..`, `-`, `~`
`echo.cpp`	16	`echo`
`pwd.cpp`	18	`pwd`
`ls.cpp`	215	`ls` with `-a`, `-l`, multiple paths, any flag order
`pinfo.cpp`	139	Reads `/proc/<pid>/stat` and `/proc/<pid>/exe`
`search.cpp`	75	Recursive file and folder search
`history.cpp`	132	20-command store, 10-command display, cross-session file
`jobs.cpp`	62	Job list and the `jobs` command
`system.cpp`	128	fork and execvp for external commands
`pipe.cpp`	192	Arbitrary-length pipelines with per-stage redirection
`redirection.cpp`	62	Extracts `<`, `>`, `>>` from a token list
`signal.cpp`	121	SIGINT, SIGTSTP, SIGCHLD handlers
`autocomplete.cpp`	172	TAB completion for commands and filenames

2

Global State

The shell keeps exactly two pieces of long-lived state. They are declared `extern` in `functions.h` and defined in `main.cpp`.

```
std::string MyHomeDir;  // dir the shell was STARTED from
std::string PrevDir;    // dir before the last successful cd
```

2.1 MyHomeDir is not $HOME

This is the subtlety that catches people reading the code for the first time.

The assignment defines the shell's home as *the directory from which the shell is invoked*, displayed as ~. It is therefore captured once, at startup, from `getcwd`:

```
char cwd[1024];
if (getcwd(cwd, sizeof(cwd)) != NULL)
    MyHomeDir = cwd;
```

Launch the shell from `/tmp/demo` and ~ means `/tmp/demo`. Typing `cd ~` goes there, not to `/home/user`.

The real `$HOME` is consulted in exactly one place in the entire program: locating the history file. History must be findable again regardless of which directory the shell was started from, so that one lookup uses `getpwuid(getuid())->pw_dir`.

2.2 PrevDir and the cd - target

`PrevDir` is initialised to `MyHomeDir` rather than left empty. That is deliberate — it makes `cd -` work on a brand-new shell instead of erroring out on the first use.

The update discipline matters more than it looks. The current directory is captured *before* `chdir` is attempted, but committed to `PrevDir` only *after* `chdir` succeeds. A failed `cd` must not corrupt the `cd -` target.

2.3 Why globals at all

The assignment forbids the environment variables `PWD` and `OLDPWD`. These two globals are the legal replacement: the shell tracks its own directory state rather than delegating to the environment.

3

The Life of an Input Line

Nine steps, from keystroke to result. This is `main.cpp` in full, described.

1. **`displayPrompt()`** builds `<user@host:dir>` fresh.
2. **`readline(prompt)`** returns a heap-allocated string, handling arrow keys, backspace, and TAB along the way.
3. **NULL check** — a null return means EOF, which is Ctrl-D. Print a newline and break the loop.
4. **`free(tmpInput)`** — readline allocates with `malloc` and hands over ownership.
5. **`isOnlyWhitespace()`** — a blank or all-spaces line is skipped without touching history.
6. **`add_to_history(input)`** — the whole raw line is stored, before any splitting.
7. **`SplitBySemicolon(input)`** turns `a ; b ; c` into three pieces.
8. **For each piece**: tokenize, skip if empty, check for `exit` or `quit`, otherwise `Runcmd(tokens)`.
9. **If the exit flag is set**, break the outer loop.

3.1 Details that matter

The prompt is rebuilt every iteration, never cached. It has to be. `cd` changes the working directory and the prompt must reflect that immediately.

History stores the raw line, before `;` splitting. Typing `ls ; pwd` creates one history entry, `ls ; pwd`, not two. This matches real bash and the examples in the specification.

`exit` breaks out of the `;` list. Running `echo one ; exit ; echo two` prints `one` and quits — `echo two` never executes. Verified.

`exit` is handled in `main`, not in `isBuiltin`. It needs to break the REPL loop, which only `main` can do. It does appear in the autocomplete word list, so TAB still completes it.

Ctrl-D is not a signal. It is end-of-file on the terminal. `readline` returns `NULL`, the loop breaks, the shell exits, and the surrounding terminal is unaffected — exactly what the specification asks for.

4

Parsing

Parsing happens in two stages with different techniques, because the two jobs have different shapes.

4.1 Stage one: splitting on semicolons

The assignment mandates the tool: *"Use 'strtok' to tokenize the command."*

```
char *buffer = new char[input.length() + 1];
strcpy(buffer, input.c_str());
char *piece = strtok(buffer, ";");
while (piece != NULL) {
    string s = piece;
    if (s.find_first_not_of(" \t") != string::npos)  // skip blanks
        parts.push_back(s);
    piece = strtok(NULL, ";");
}
delete[] buffer;
```

Three properties of `strtok` drive this code:

It mutates its input. It writes a NUL byte over each delimiter it finds. You cannot hand it `input.c_str()`, which points into the `std::string`'s own buffer and is `const`. Hence the explicit heap copy and the matching `delete[]`.

It keeps a static internal pointer. It is not reentrant and not thread-safe. That is fine in a single-threaded shell that finishes the whole split before doing anything else, but it is the standard criticism of `strtok` and worth being able to state.

It collapses consecutive delimiters. `a ;; b` yields two pieces, not three with an empty middle. The blank-piece filter is belt-and-braces for the `a ;  ; b` case, where the middle piece is spaces rather than empty.

4.2 Stage two: the character scanner

`strtok` cannot do the second job. Splitting on delimiters cannot turn `ls>out.txt` into three tokens, because there is no delimiter between `ls` and `>`. A character-level scanner can.

The scanner walks the string one character at a time, carrying two state variables:

```
bool inQuote = false;
char quoteChar = '\0';
```

Each character falls into one of four cases:

Case	Action
`"` or `'`	Opening: set the quote state. Closing (matching character): clear it. A different quote character while already inside: keep it literally
Whitespace, not quoted	Token separator. Push if non-empty, otherwise ignore — this is what collapses runs of spaces and tabs
`|`, `<`, `>`, `&`, not quoted	Push any pending token, then push the symbol as its own token. `>` peeks ahead one character for `>>`
Anything else	Append to the current token

The `>>` lookahead is the fiddly part: check `i + 1 < input.length()` *before* indexing, then increment `i` to consume the second character.

Why symbols become standalone tokens. It makes every later stage trivial. Redirection parsing compares whole strings against `"<"`, `">"`, and `">>"`. Pipeline splitting looks for `"|"`. No stage downstream ever does substring searching.

4.3 Quote handling is deliberately shallow

The specification is explicit: *"You do not need to handle escape flags and quotes. You can print the string as it is."*

So quotes group tokens and are then stripped. `echo "abc 'ac' abc"` produces `abc 'ac' abc` — verified. The inner single quotes survive because they are the non-matching type while inside double quotes.

Two consequences follow, both permitted:

`echo ""` prints an empty line rather than passing an empty argument, because the closing-quote branch pushes the token only if it is non-empty. Verified: `echo "" end` prints `end`.

Quotes cannot protect a semicolon. Stage one runs before stage two and has no quote awareness, so `echo "a;b"` splits into `echo "a` and `b"`, printing `a` and then failing on `b`. General note 10 of the specification permits this directly: *"The symbols <, >, », &, |, ; , - would always correspond to their special meaning and would not appear otherwise, such as in inputs to echo."*

5

The Executor

5.1 Recognising a builtin

`isBuiltin` is a flat string comparison against eight names:

`cd` `echo` `pwd` `ls` `pinfo` `search` `history` `jobs`

`runBuiltin` is the matching if-else dispatch. `exit` and `quit` are deliberately absent — they are handled in `main` because they must break the loop.

5.2 The router

`Runcmd` decides what happens to one command. The order of its checks is load-bearing:

```
1. empty?                    → return
2. tokens.back() == "&"?     → background = true, pop it off
3. PipelineHandle(tokens)    → if true, a pipe was found and handled;
↪   return
4. isBuiltin(tokens[0])?     → parse redirection, run in-process
5. otherwise                 → systemCommand(tokens, background)
```

The pipeline check must precede the builtin check. `ls | wc` has a builtin as its first token but must go down the pipeline path, not the in-process path. Reversing these two lines would break every pipeline that starts with a builtin.

`&` is recognised only as the final token. `sleep 5 & echo hi` does not behave as bash would. The specification's examples only ever show a trailing `&`, and general note 10 again covers the symbol's position.

5.3 Redirection for builtins

A builtin has no child process to discard, so the shell must restore its own file descriptors afterwards. Without that, `echo hi > f.txt` would leave the shell's stdout permanently attached to `f.txt`, and the prompt would vanish into the file.

```
int savedIn  = dup(STDIN_FILENO);   // bookmark originals
int savedOut = dup(STDOUT_FILENO);

// ... open target, dup2(fd, STDOUT_FILENO), close(fd) ...

if (ok) runBuiltin(cmdtokens);

cout.flush();                       // essential — see below

dup2(savedIn,  STDIN_FILENO);       // restore
dup2(savedOut, STDOUT_FILENO);
close(savedIn); close(savedOut);    // don't leak the bookmarks
```

> **The `cout.flush()` is not optional.** `std::cout` maintains a userspace buffer; data sits there until the buffer fills or is explicitly flushed. Restoring the original stdout *before* flushing sends those buffered bytes to the terminal instead of the file — the output appears on screen and the file ends up empty. Flushing while descriptor 1 still points at the file is what makes the redirection land.

An `ok` flag guards the sequence. If opening the input file fails, the command is skipped entirely, but the restore still runs.

6

The Builtin Commands

6.1 The prompt

Three lookups, each with a fallback, and nothing hard-coded — the specification forbids hard-coding the user and system names.

Data	Call	Fallback
Username	`getpwuid(getuid())` then `pw->pw_name`	`"unknown"`
Hostname	`gethostname(buf, sizeof buf)`	`"unknown"`
Directory	`getcwd(buf, sizeof buf)`	`"?"`

The result is assembled as `<user@host:dir>`.

6.1.1 The tilde substitution has a real bug guard

```
if (currDir.compare(0, MyHomeDir.length(), MyHomeDir) == 0) {
    if (currDir.length() == MyHomeDir.length() ||
     ↪  currDir[MyHomeDir.length()] == '/')
        currDir.replace(0, MyHomeDir.length(), "~");
}
```

A naive prefix test alone is wrong. If the shell's home is `/home/u/test` and you move to `/home/u/test2`, the prefix matches and the prompt would read `~2` — nonsense. The second condition requires the match to end either at the end of the string or at a `/`, so only the home directory itself or something genuinely inside it is abbreviated.

One more detail: `gethostname` does not guarantee NUL-termination when it truncates, so the code forces a terminator at the end of the buffer.

6.2 cd

The argument count is checked first. More than one argument prints exactly `Invalid arguments`, the string the specification requires.

Input	Resolves to
(none)	`MyHomeDir`
`~`	`MyHomeDir`
`~/foo`	`MyHomeDir + "/foo"`
`-`	`PrevDir`, **and prints the path** — bash does this too
`. .. /abs rel`	Passed through untouched; `chdir` handles them

The ordering discipline described in Chapter 2 applies here: capture with `getcwd` before the attempt, commit to `PrevDir` only after success. Errors are reported through `perror`, so a missing directory produces `cd failed: No such file or directory`.

6.3 echo

Sixteen lines. It joins arguments with a single space and adds a newline. Because the tokenizer already collapsed runs of whitespace, `echo hello  world` prints `hello world` — verified. Multi-line strings and environment variable expansion are explicitly out of scope.

6.4 pwd

`getcwd` into a 1024-byte buffer, print it, or report through `perror`. It always yields the absolute path from the root, which is what the specification asks for after `cd …`

6.5 ls

The largest builtin, and the one with the most interesting design problem.

6.5.1 Building the permission string

The first character comes from `S_ISDIR` or `S_ISLNK`; the remaining nine are bit tests against `S_IRUSR` through `S_IXOTH`, producing strings like `-rw-r--r--`.

6.5.2 Why -l needs two passes

Listing a directory means `opendir`, `readdir`, `closedir`, collecting names into a vector and sorting them — `readdir` returns entries in directory order, which is arbitrary, while real `ls` sorts alphabetically. Hidden entries are skipped unless `-a` was given.

For `-l`, the implementation makes two passes over the entries:

- **Pass one** computes the block total and the maximum width of the link-count, user, group, and size columns.
- **Pass two** prints each line using those widths through `setw`.

The reason is straightforward once stated: you cannot know the widest username until you have called `stat` on every entry. Without the measuring pass the columns would not align. Real `ls` works the same way.

> **The block total.** `st_blocks` is defined by POSIX in 512-byte units, while `ls` reports 1 K blocks. Hence the division by two when accumulating the total.

`lstat` is used rather than `stat`, so a symbolic link is reported as a link rather than as whatever it points at.

6.5.3 Argument handling

Flags and paths may appear **in any order**, as the specification requires with its `ls -l dir1 -a dir2` example. Bundled flags are split per character, so `-la`, `-al`, and `-a -l` are all equivalent. An unknown flag reports `ls: It is invalid option -- 'x'` and skips only that argument. Tilde paths are expanded here too. With no path given, the default is `.`. With several paths, each gets a `path:` header and a blank line between, matching real `ls`.

The output was **verified byte-identical to the system `ls -l`**, including the total line and column alignment.

6.6 pinfo

`popen` is forbidden, so every value is read from the `/proc` filesystem directly.

With no argument the target is `getpid()`, the shell itself. With one argument, the string is validated digit by digit with `isdigit` before `stoi`, so `pinfo abc` reports `pinfo: PID is invalid` instead of throwing an exception.

6.6.1 The classic /proc parsing trap

Field two of `/proc/<pid>/stat` is the executable name in parentheses, and **it may contain spaces and parentheses of its own**. A process named `my prog (test)` destroys naive whitespace splitting. The fix:

```
size_t bracket = line.rfind(')');       // rfind — the LAST ')', not the
↪  first
istringstream iss(line.substr(bracket + 1));
```

Everything after the final `)` is clean, space-separated numeric data. The index mapping is then documented in the code: `fields[0]` corresponds to stat field 3, so **file field N is `fields[N-3]`**.

Wanted	stat field	Code index
State	3	`fields[0]`
pgrp	5	`fields[2]`
tpgid	7	`fields[4]`
vsize	23	`fields[20]`

A `fields.size() < 21` guard prevents an out-of-range read on a malformed line.

6.6.2 The foreground marker

The `+` suffix is appended when `tpgid` equals `pgrp` — the process group owning the terminal is this process's own group, meaning it is in the foreground. That yields `R+` and `S+` as the specification requires.

6.6.3 The executable path

`readlink` on `/proc/<pid>/exe` gives the real path, but fails with `EACCES` for processes owned by other users. The fallback reads `/proc/<pid>/cmdline`, which is NUL-separated — `getline(f, s, '\0')` takes just the first field — and failing that, reports `N/A`. The result is passed through a helper that abbreviates the shell's home as `~`.

6.7 search

A recursive depth-first search from the current directory, printing `True` or `False`.

- `.` and `..` are skipped, or the recursion would never terminate.

- Matching is on name alone, so it finds both files and folders as required.
- It returns as soon as it matches, and `closedir` is called on every exit path.
- Unreadable directories make `opendir` fail and are skipped rather than crashing.

> **`lstat`, not `stat` — and this one is not cosmetic.** `stat` follows symbolic links. A link pointing at an ancestor directory creates a cycle, and the search runs forever. `lstat` sees the link as a link and does not descend into it.

6.8 history

The constants are twenty stored, ten displayed, in a file named `.myshell_history`.

This is the **one place the real `$HOME` is used**, via `getpwuid(getuid())->pw_dir`. History has to be found again regardless of the directory the shell was launched from, so it cannot live under `MyHomeDir`.

At startup, `LoadHistory` reads the file, trims to the newest twenty, and pushes every entry into readline's own list with `add_history` — which is what makes the UP and DOWN arrows work across sessions.

On each command, `add_to_history` trims surrounding whitespace, ignores empty input, appends, evicts the oldest while over the limit, and **saves the file immediately**. Saving on every command rather than at exit means history survives a crash or a `kill`, not just a clean exit.

On display, the default is ten, or `history <n>` for a specific count. The argument is validated with `stoi(s, &pos)` plus a `pos != length` check, so `12abc` is rejected. Non-positive values are rejected and values above twenty are clamped.

Verified: twenty-five commands entered leaves exactly twenty stored with the oldest evicted; bare `history` shows ten; entries persist into a new session; and `exit` is recorded, matching the specification's own example.

6.8.1 Two stores kept in sync

There are actually two history stores, and understanding why is worth a moment.

1. **`CmdHistory`**, the shell's own vector, is what the `history` command prints and what gets written to disk.
2. **Readline's internal list**, fed by `add_history`, is what the arrow keys traverse.

`LoadHistory` clears and repopulates the second from the first at startup; `add_to_history` writes to both. Keeping them in step is why arrow-key recall and the `history` command never disagree.

6.9 jobs

Not a numbered requirement, but necessary to make Ctrl-Z observable. A static vector holds `{pid, name, stopped}` records.

`add_job` updates in place when the PID is already present, so pressing Ctrl-Z on an existing background job flips its state rather than creating a duplicate row. Output is formatted `[1] Stopped  name [pid]`. The index is the vector position plus one, so numbers shift after a removal — simpler than bash's persistent job IDs, and sufficient here.

7

External Commands

`systemCommand` is the fork-and-exec core.

7.1 Before the fork

Redirection is parsed, syntax errors abort, and — a deliberate touch — **the input file is test-opened in the parent**:

```
int testfd = open(redir.inputfile.c_str(), O_RDONLY);
if (testfd < 0) { perror(...); return; }
close(testfd);
```

The specification says *"An error message should be displayed if the input file does not exist."* Checking in the parent means the shell reports the error before any process is created, so no stray child ever appears in the job list. The child re-opens the file for real afterwards.

Arguments are converted into a NUL-terminated `char*` array for `execvp`. The `const_cast` is needed because `execvp` takes `char* const[]` for historical C reasons; it does not modify the strings.

7.2 Inside the child

```
setpgid(0, 0);              // own process group
signal(SIGINT,  SIG_DFL);   // restore default dispositions
signal(SIGTSTP, SIG_DFL);
// ... apply redirections via dup2 ...
execvp(cargs[0], cargs.data());
perror(cmdtokens[0].c_str());   // only reached if exec FAILED
_exit(1);
```

Three decisions here need justification.

`setpgid(0, 0)` puts the child in its own process group. The terminal delivers Ctrl-C and Ctrl-Z to the entire foreground process group. Without this call the child shares the shell's group and the signal would strike the shell as well. Isolating the child lets the shell's own handler decide what to forward.

Signal dispositions are reset. `fork` inherits the parent's handlers, so without this the child would run the *shell's* SIGINT handler. It must behave normally instead.

`_exit(1)`, not `exit(1)`. `exit` flushes stdio buffers and runs atexit handlers. Those buffers were copied from the parent, so flushing them reprints the parent's pending output from the child. `_exit` terminates immediately without touching them.

Reaching the line after `execvp` at all means the exec failed — a bad command name, or a file that is not executable — so `perror` reports, for example, `foo: No such file or directory`.

7.3 Inside the parent

`setpgid(pid, pid)` is called **again**. This looks redundant and is intentional: it removes a race. Whichever of the two processes is scheduled first, the group is established before it can matter. The duplicate call simply re-sets the same value and is harmless.

Foreground path:

```
set_foreground_pid(pid);
set_foreground_name(cmdtokens[0]);
waitpid(pid, &status, WUNTRACED);   // also returns when STOPPED
set_foreground_pid(-1);
if (WIFSTOPPED(status)) { add_job(pid, name, true); /* print message */ }
```

`WUNTRACED` is the critical flag. Without it, `waitpid` returns only when the child *terminates*, so pressing Ctrl-Z would leave the shell hanging forever on a stopped child.

Background path: no wait at all. Record the job, print the PID as the specification requires, and return to the prompt immediately. Reaping happens later, in the SIGCHLD handler.

8

Redirection

The redirection parser walks the token vector. For each of `<`, `>`, and `>>` it takes the *next* token as the filename, with a bounds check first — a trailing operator produces `Syntax error: file name is missing after '>'` and sets an error flag. Every token that is neither an operator nor a filename is copied into a clean argument list, which replaces the original vector. The caller therefore receives arguments with the redirection already stripped out.

Later operators overwrite earlier ones, so `> a > b` ends up writing to `b`. That matches bash closely enough for the cases under test.

8.1 The open flags

Operator	Flags	Meaning
`<`	`O_RDONLY`	Must exist, otherwise error
`>`	`O_WRONLY | O_CREAT | O_TRUNC`, mode 0644	Create, or empty an existing file
`>>`	`O_WRONLY | O_CREAT | O_APPEND`, mode 0644	Create, or add to the end

Mode 0644 is mandated by the specification and was **verified** — test files came out with exactly those permissions.

`O_APPEND` deserves a proper understanding: it makes every write *atomically* seek to the current end of file. That is materially different from opening and seeking once, and it is what makes appending safe when two processes write to the same file concurrently.

8.2 The dup2 pattern

```
int fd = open(file, flags, 0644);
dup2(fd, STDOUT_FILENO);   // fd 1 -> the file; old fd 1 closed
close(fd);                 // the extra descriptor is no longer needed
```

`dup2(old, new)` closes `new` if it is open, then makes it a copy of `old`. Both descriptors then refer to the same open file description, sharing the file offset and status flags. Closing the original afterwards is not merely tidiness — leaving it open leaks a descriptor for the lifetime of the process.

For external commands this happens in the child after `fork`, so the shell's own descriptors are never touched. For builtins, the save-and-restore sequence from Chapter 5 is required instead.

9

Pipelines

`PipelineHandle` returns `false` immediately when there is no `|`, letting the router continue down its normal path. Otherwise it takes over completely and returns `true`.

9.1 The six steps

Split on `|`. An empty segment on either side is a syntax error, catching `cat |`, `| wc`, and `a || b`.

Parse redirection for every stage before forking anything. This is deliberate: a syntax error in stage three is caught before any process exists, so there are no orphans to clean up.

Create `n-1` pipes into a flat vector of size `2(n-1)`. Pipe *i* occupies index `2i` for its read end and `2i+1` for its write end. This flat layout is why the child code reads `pipefds[(i-1)*2]` for "the previous stage's read end" and `pipefds[i*2+1]` for "my write end".

Fork each stage. In each child:

```
input:   file from '<' if present  →  else pipe from previous stage (when i >
↪  0)
output:  file from '>' if present  →  else pipe to next stage       (when i <
↪  n-1)
then:    close ALL 2(n-1) pipe descriptors
then:    builtin  → runBuiltin, flush, _exit(0)
         external → execvp
```

File redirection deliberately takes precedence over the pipe. In `ls | grep txt > out.txt`, the output of `grep` goes to the file rather than onward. This matches bash and is what the specification's ninth requirement asks for.

The parent closes every single pipe descriptor. This is the most important line in the file:

```
for (int j = 0; j < 2 * pipecount; j++) close(pipefds[j]);
```

> **Why the shell hangs without it.** A reader sees end-of-file only when *every* write end of the pipe has been closed. After forking, the parent still holds copies of all of them. If the parent does not close its copies, `wc` never sees EOF, never exits, and the shell waits forever in `waitpid`. The children closing their own copies is not sufficient — the parent's must go too. This is the classic pipeline bug, and the reason a dedicated hang test exists in the testing guide.

Wait for all stages. The *last* stage is registered as the foreground process, so Ctrl-C reaches it and the SIGCHLD handler stays out of the way. All PIDs are then waited on in order.

Pipelines of arbitrary length are supported, as required. Two- and three-stage pipelines were verified, along with redirection at both ends.

10

Signals and Job Control

Three handlers are installed through `sigaction` rather than `signal`, because `sigaction` has portable, well-defined semantics and allows explicit flags.

The module keeps three pieces of state:

```
static pid_t PIDfg = -1;                        // current foreground pid
static string NameFg = "";                      // its name, for messages
static volatile sig_atomic_t waitingFg = 0;  // 1 while waiting on fg
```

> **Why `volatile sig_atomic_t`.** It is the only type guaranteed safe to touch from both a signal handler and normal code. `volatile` stops the compiler caching the value in a register and missing a change made by the handler; `sig_atomic_t` guarantees the read or write completes in one uninterruptible step.

10.1 Ctrl-C, or SIGINT

```
if (PIDfg > 0) kill(PIDfg, SIGINT);
// else: do nothing — the shell must survive
```

This matches the specification exactly: *“This should have no effect on the shell if there is no foreground process running.”*

10.2 Ctrl-Z, or SIGTSTP

```
if (PIDfg > 0) { kill(PIDfg, SIGTSTP); add_job(PIDfg, NameFg, true); }
```

The child stops. The blocked `waitpid` with `WUNTRACED` returns, `WIFSTOPPED` is true, and the shell prints that the job has been stopped and sent to the background. It then appears as `Stopped` in the `jobs` listing.

10.3 SIGCHLD, or reaping the background

The flags are `SA_RESTART | SA_NOCLDSTOP`.

`SA_RESTART` automatically restarts interrupted slow system calls. Without it, every background process exit would make `readline` fail with `EINTR` and corrupt the prompt.

`SA_NOCLDSTOP` suppresses SIGCHLD when a child merely *stops*. Ctrl-Z is already handled by the SIGTSTP path, so this prevents a duplicate notification.

The handler body:

```
if (waitingFg) return;                                  // (1)
while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {   // (2)
    string msg = "\n" + name + " with pid " + to_string(pid) + " exited\n";
    write(STDOUT_FILENO, msg.c_str(), msg.length());  // (3)
    remove_job(pid);
}
```

(1) The `waitingFg` guard prevents a race in which the handler reaps a *foreground* child before the blocking `waitpid` in `system.cpp` can. That would leave the blocking call returning `-1` with `ECHILD` and the shell confused about a job that had already finished.

(2) `WNOHANG` makes `waitpid` return zero instead of blocking when no child has changed state — essential, because a signal handler must never block. The surrounding `while` loop matters just as much: standard signals are **not queued**, so several children exiting at nearly the same moment may produce only one SIGCHLD delivery. The handler must drain every finished child in a single pass.

(3) `write`, not `cout`. Only async-signal-safe functions are legal inside a handler. `write` is a thin syscall wrapper on the POSIX safe list; `std::cout` is not — it can allocate, and it holds internal state that may be mid-update when the signal arrives, risking deadlock or corrupted output.

10.4 Ctrl-D

Not a signal at all, as noted in Chapter 3 — it is end-of-file on the terminal, handled by the `NULL` return from `readline`.

11

Autocomplete

Built on readline's completion hooks. Setup assigns a custom completion function and binds the TAB key.

11.1 Deciding what to complete

The code scans the line buffer from the start up to the current word. If everything before it is spaces and tabs, this is the first word and **commands** are completed; otherwise **filenames** are. Setting `rl_attempted_completion_over` stops readline from appending its own default filename completion afterwards.

11.2 Where the candidates come from

Commands are the builtin names plus a full scan of every directory in `$PATH`, keeping only entries where `access(full, X_OK)` succeeds, then sorted and de-duplicated — the same program frequently appears in several PATH directories. The scan is **lazy**, triggered on the first TAB and cached behind a flag, because walking all of PATH is expensive and most sessions never need it.

Filenames come from `readdir` on the current directory, prefix-matched, with a `/` appended to directories for readability.

11.3 The generator protocol

This is the part that looks strange if you have not met readline before. Readline calls the generator repeatedly for a single completion: a `state` of zero means "new completion, reset yourself", any later call means "give me the next match", and returning `NULL` means "no more". That is why the generators keep `static` index variables across calls.

> Each returned string must be `strdup`’d, because **readline takes ownership and frees it**. Returning a pointer into a `std::string` would cause a double free.

Readline itself then computes the common prefix and decides whether to insert it or list the alternatives — which is why a single match completes inline while multiple matches produce a listing, exactly as the specification describes.

12

System Call Reference

Call	Used in	Purpose
`fork`	system, pipe	Create a child process
`execvp`	system, pipe	Replace the child image; `p` searches `$PATH`, `v` takes an argv array
`waitpid`	system, pipe, signal	Reap children; `WUNTRACED` for stops, `WNOHANG` in the handler
`_exit`	system, pipe	Child exit without flushing inherited buffers
`pipe`	pipe	Create the connected descriptor pair
`dup`	executor	Bookmark the shell's own stdin and stdout
`dup2`	throughout	Point a specific descriptor at another open file
`open`, `close`	redirection paths	File access for `<`, `>`, `>>`
`chdir`	cd	Change the working directory
`getcwd`	main, display, pwd, cd, search	Absolute current directory
`stat`, `lstat`	ls, search, autocomplete	File metadata; `lstat` does not follow links

Call	Used in	Purpose
`opendir`, `readdir`, `closedir`	ls, search, autocomplete	Directory iteration
`getpwuid`, `getgrgid`	display, ls, history	UID and GID to names, and the real `$HOME`
`getuid`, `getpid`	display, pinfo	Process identity
`gethostname`	display	Hostname for the prompt
`readlink`	pinfo	Resolve `/proc/<pid>/exe`
`kill`	signal	Forward SIGINT and SIGTSTP
`sigaction`	signal	Install handlers with explicit flags
`setpgid`	system, pipe	Isolate children into their own process groups
`access`	autocomplete	`X_OK` test when scanning PATH
`write`	signal	Async-signal-safe output inside a handler
`perror`	throughout	Translate `errno` into a readable message

Libraries. GNU readline provides line editing, arrow-key history, and TAB completion, linked with `-lreadline`. It is the only external dependency. The ncurses library is banned by the specification and is not used; the C++ `filesystem` library is banned and is not used.

13

Twelve Concurrency Traps

These are the bugs that make a shell genuinely hard to write. Each was encountered, diagnosed, and fixed.

Trap	Symptom	Fix
Parent keeps pipe descriptors open	Pipeline hangs forever; `wc` never prints	Parent closes **all** `2(n-1)` descriptors after forking
SIGCHLD reaps a foreground child	`waitpid` returns `-1`; job state corrupted	The `waitingFg` flag makes the handler return early
`cout` inside a signal handler	Possible deadlock or interleaved garbage	Use `write`, which is async-signal-safe
Builtin redirection not flushed	File ends up empty; text appears on screen	`cout.flush()` **before** restoring descriptors
`setpgid` race between parent and child	Signals occasionally reach the wrong group	Call it in **both** processes
Child inherits the shell's handlers	Ctrl-C runs shell logic inside the child	Reset dispositions to `SIG_DFL` after fork
`stat` following a symlink loop	`search` never terminates	Use `lstat` instead
Prefix match on the home directory	`/home/u/test2` displays as `~2`	Require end-of-string or `/` after the prefix
Spaces or parentheses in the `/proc` comm field	`pinfo` reads the wrong fields	Parse only after `rfind(')')`
Missing `WUNTRACED`	Shell hangs when a job is stopped	`waitpid(pid, &st, WUNTRACED)`

Trap	Symptom	Fix
Signals are not queued	Some background exits never reported	Drain with `while (waitpid(... WNOHANG) > 0)`
`exit()` after a failed exec	Parent’s buffered output printed twice	Use `_exit()` in the child

14

Test Evidence

Everything below was run against a clean rebuild. Nothing in this chapter is inferred from reading the source.

Build. `make clean && make` succeeds with **zero warnings** under `-Wall -Wextra -std=c++11`.

Area	Result
Prompt, tilde abbreviation, `cd` reflected live	Pass
`pwd`, and `echo` with collapsed spaces	Pass
`echo "abc 'ac' abc"` produces `abc 'ac' abc`	Pass — matches the specification
`ls`, `ls -a`, `ls -l`, `ls -la sub`	Pass — **byte-identical to system `ls`**, total included
`ls nonexist`	Pass — `ls: It cannot access nonexist: No such file or directory`
`cd` with `.`, `..`, `-`, `~`; `cd a b` rejected	Pass — prints `Invalid arguments`
`search f.txt` and `search zzz`	Pass — `True` and `False`
`pinfo` status, virtual memory, tilde path	Pass
`>`, `>>`, `<`, and combined `sort < f > s`	Pass
Output file permissions	Pass — **0644 confirmed via `stat`**
`cat < missing.txt`	Pass — error raised before any process spawns

Area	Result
`echo x >`	Pass — `Syntax error: file name is missing after '>'`
Two- and three-stage pipelines	Pass — no hang
`ls | grep txt > g.txt ; cat g.txt`	Pass — redirection inside a pipeline, plus `;`
Semicolon separation with ragged spacing	Pass
Background `&` prints the PID, prompt returns at once	Pass
`exit` mid-list halts the remaining commands	Pass
History: 25 entered, 20 stored, oldest evicted	Pass
History: bare `history` shows 10; `history 3` shows 3	Pass
History persists into a new session	Pass
Forbidden `popen`, `pclose`, `system`	Pass — **grep confirms absent**

Not covered by these runs. Signals (Ctrl-C, Ctrl-Z, Ctrl-D) and TAB completion require an interactive terminal and were not exercised by piped input. Their code paths are documented in Chapters 10 and 11, and the accompanying testing guide covers them manually in its sections 10 and 11.

15

Known Limitations

An honest list. Most are explicitly permitted by the assignment, and the permitting clause is cited where it applies.

1. A semicolon inside quotes still splits. `echo "a;b"` breaks into two commands, printing `a` and then failing on `b`, because stage-one splitting runs before quote parsing. *Permitted by general note 10.*

2. `&` is recognised only as the final token. `sleep 5 & echo hi` does not background and continue. Only the trailing form appears in the specification.

3. A backgrounded pipeline still blocks. `sleep 2 | cat &` strips the `&`, but the pipeline handler always waits for every stage. **Measured at 2.01 seconds elapsed**, confirming the block rather than merely suspecting it. The specification never combines `&` with `|`.

4. Builtins in a pipeline lose their side effects. `cd sub | cat` leaves the shell's directory unchanged — verified. This is correct POSIX behaviour and bash does the same, but it is worth stating explicitly.

5. `echo ""` prints an empty line rather than passing an empty argument.

6. No `fg` or `bg` commands. A stopped job can be listed but not resumed. Not required — the tenth requirement asks only for the three keystrokes.

7. Job numbers renumber after a job is removed, unlike bash's stable job IDs.

8. No glob expansion, no variable expansion, no escape sequences. `*.txt` and `$HOME` are not expanded. All out of scope per the specification.

9. Fixed 1024-byte path buffers for `getcwd`. Adequate in practice; a pathologically deep path would fail the call, which is then handled gracefully rather than crashing.

10. `ls` prints one entry per line rather than in columns. Explicitly allowed: *"outputting the entries in a single column is fine."*

11. **Duplicate redirection operators** keep the last one; bash's more elaborate ordering rules are not modelled.

16

Rule Compliance

Requirement from the specification	Status
C++ only	Met
Modular code	Met — 18 files, one command per file
`strtok` used for tokenizing	Met — in the semicolon splitter
makefile with proper flags and linker options	Met — `-Wall -Wextra -std=c++11 -g`, `-lreadline`
README.md included	Met
No `popen`, `pclose`, or `system()`	Met — grep-verified absent
No ncurses	Met
No `filesystem` library	Met
No `PWD` or `OLDPWD` environment variables	Met — replaced by the two globals
`pipe()` allowed and used	Met
No `execvp` for the shell's own builtins	Met — all eight run in-process
`exec` family for other commands	Met — `execvp`
`fork` and `wait` for children	Met
Signal handlers for background exits	Met — SIGCHLD
`perror` for error handling	Met — used throughout
Output files created with mode 0644	Met — verified
Random spaces and tabs handled	Met — verified
Background PID printed	Met

Requirement from the specification	Status
Multiple simultaneous background processes	Met — the job list is a vector
Any number of pipes	Met — `n-1` pipes for `n` stages
History tracked across sessions	Met — verified
History stores 20, displays 10	Met — verified

17

Viva Question Bank

Why must `cd` be a builtin? `fork` copies the process state. A child's `chdir` changes only its own copy and dies with it. The parent shell — the process that actually needs to move — is unaffected.

What exactly does `fork()` return? Zero in the child, the child's PID in the parent, and `-1` on failure. That is how one line of code takes two different branches.

Why `_exit` rather than `exit` in a child after a failed exec? `exit` flushes stdio buffers and runs atexit handlers. Those buffers were copied from the parent, so flushing them reprints the parent's pending output. `_exit` terminates immediately without touching them.

What does `dup2(fd, 1)` do? It closes descriptor 1 if open, then makes it refer to the same open file description as `fd`. Both then share the file offset and status flags.

Why must the parent close its pipe descriptors? A reader sees end-of-file only when *all* write ends are closed. The parent holds a copy of every end after forking. Leaving them open means `wc` waits for input that never ends, and the shell blocks forever in `waitpid`.

Why is `printf` or `cout` unsafe inside a signal handler? Neither is async-signal-safe. A signal can arrive while the stream's internal lock or buffer is mid-update; re-entering can deadlock or corrupt output. `write` is a direct syscall wrapper and appears on the POSIX safe list.

What is `WNOHANG` for? It makes `waitpid` return zero immediately instead of blocking when no child has changed state — essential in a handler, which must never block.

Why loop the `waitpid` inside the SIGCHLD handler? Standard signals are not queued. Three children exiting at nearly the same time may produce a single SIGCHLD delivery, so the handler must drain every reapable child before returning.

What does `WUNTRACED` add? It makes `waitpid` return when the child is *stopped*, not only when it terminates. Required for Ctrl-Z; without it the shell hangs on a stopped job.

Why call `setpgid` in both the parent and the child? To close a scheduling race. Either process may run first, so setting the group in both guarantees it is established before any signal could be delivered to the wrong group.

Why `lstat` instead of `stat` in `search`? `stat` follows symbolic links. A link pointing at an ancestor directory creates a cycle and the recursion never ends. `lstat` reports the link itself, so the search does not descend into it.

How does `pinfo` decide to print the `+`? It compares stat field 7, `tpgid` — the process group owning the terminal — against field 5, `pgrp`. Equality means this process group is the terminal's foreground group.

Why parse `/proc/<pid>/stat` starting from the last `)`? Field 2 is the executable name in parentheses and may itself contain spaces and parentheses. Splitting on whitespace from the beginning misaligns every subsequent field. Everything after the final `)` is clean numeric data.

Why does `ls -l` need two passes? Column widths depend on the widest value present, and you cannot know the longest username or size until every entry has been stat'd. The first pass measures and totals the blocks; the second prints with `setw`.

Why divide `st_blocks` by two? `st_blocks` counts 512-byte units by POSIX definition, while `ls` reports 1 K blocks.

Why is `waitingFg` declared `volatile sig_atomic_t`? `volatile` prevents the compiler from caching it in a register and missing a change made by the handler. `sig_atomic_t` guarantees reads and writes happen in a single uninterruptible step.

Where does `~` point in this shell, and why? To the directory the shell was launched from, not `$HOME` — the assignment defines it that way. The real `$HOME` is used only to locate the history file, so history is found no matter where the shell starts.

What happens on Ctrl-D? It is end-of-file, not a signal. `readline` returns `NULL`, the loop breaks, the shell exits, and the surrounding terminal is unaffected.

Why does `cd sub | cat` not change the directory? Every pipeline stage runs in a forked child, so the builtin executes there and `chdir` affects only that short-lived process. Bash behaves identically.

What is the difference between a process group and a session, and why does the shell care? A process group is the unit of signal delivery — the terminal sends Ctrl-C to every process in the foreground group. A session is a larger collection of process groups attached to one controlling terminal. The shell cares because putting each child

in its own group is what lets it forward signals selectively instead of having them strike the shell too.